Project 16: Shelf Sense: Offline Inventory & Barcode Scanner

Domain: Small Business / Retail	**Difficulty:** Intermediate → Advanced

1. One-Line Concept

An installable, offline-first stock keeper for small shops that scans barcodes with the device camera and warns before items run out.

2. Real-World Problem

Small retailers often rely on paper or chat messages, leading to stock-outs and lost sales. Shop owners use it on a phone to track stock, record sales and see what to reorder.

3. Core Features

- Products with SKU / barcode, price, stock, reorder level, supplier and photo.
- Stock-in / stock-out movements recorded in an immutable ledger.
- Barcode scanning via camera where supported, with manual entry fallback.
- Low-stock and expiry alerts.
- Quick-sale mode: tap product tiles to build a bill and reduce stock.
- Search, filter and sort; virtualised list for thousands of items.
- CSV import / export and daily sales / profit summary chart.
- Installable app (manifest + service worker) with offline indicator and storage-usage meter.

4. HTML Skills

- Web App Manifest and meta tags
- `<video>` for the camera preview
- `<dialog>` and `<form>` flows
- `<table>` and list semantics
- Live regions for scan results and sync status

5. CSS Skills

- Mobile-first design with large touch targets
- Grid / Flexbox product tiles
- Bottom navigation and safe-area insets
- Skeleton and status indicators
- `prefers-color-scheme` theme

6. JavaScript Skills

- IndexedDB schema, indexes and migrations
- Transactional ledger updates
- Service worker caching strategies
- Camera stream lifecycle management
- Virtual list rendering
- CSV parsing / generation

7. Browser APIs / Web APIs

- `IndexedDB` – products and ledger
- `MediaDevices.getUserMedia` + `BarcodeDetector` – scanning (progressive enhancement)
- `Service Worker` + `Cache API` + Web App Manifest – offline / installable (requires HTTPS or localhost)
- `File API` – CSV import / export and product photos
- `navigator.onLine` events and `StorageManager.estimate()`
- `Notification` API – optional low-stock alerts

8. Data / Storage

IndexedDB is the source of truth with stores `products`, `movements` and `settings`. The service worker caches the app shell (cache-first). Mock JSON `data/sample-products.json` seeds a demo shop. No backend.

9. UI/UX Requirements

Designed for one-handed use in a busy shop: big buttons, instant feedback, sound / vibration on scan. Offline should feel normal, not like an error. Always show what happened (stock before / after) and allow quick corrections.

10. Responsive Requirements

Device	Expected behaviour
Mobile	Primary target: bottom tabs (Sell / Stock / Scan / Reports).
Tablet	Split view with list and product detail.
Desktop	Back-office layout with table, filters and charts; keyboard shortcuts for bulk entry.

11. JavaScript Logic Challenges

1. Design an IndexedDB schema whose ledger always matches current stock.
2. Choose service-worker strategies (cache-first shell, stale-while-revalidate data).
3. Run a barcode detection loop efficiently and release the camera correctly.
4. Render very long product lists smoothly with virtualisation.

12. Advanced Features

1. Generate and print barcode labels on Canvas.
2. Background 'export to CSV' reminder when storage is high.
3. Supplier reorder list grouped by supplier and copyable as a message.

13. Difficulty

Intermediate → Advanced

14. Resume Value

A genuine offline-first product with camera input, service worker and IndexedDB signals advanced frontend capability — often reserved for much more experienced developers.

Skills Demonstrated: IndexedDB · Service Workers & PWA · Camera / Media APIs · Performance Optimization · Offline-First Design · Virtual Lists · Responsive (mobile-first) Design · Error Handling.

15. Suggested Folder Structure

```
shelf-sense/
├── index.html
├── manifest.webmanifest
├── sw.js
├── css/
│   └── style.css
├── js/
│   ├── main.js
│   ├── db.js
│   ├── ledger.js
│   ├── scanner.js
│   ├── virtual-list.js
│   ├── csv.js
│   └── ui.js
├── data/
│   └── sample-products.json
└── assets/
    └── icons/
```